

Data-Centric Systems Engineering (DCSE) based on Semantic Digital Thread (SeDT)

Author: Hooshmand, Yousef

Affiliation: NIO

DOI: 10.5281/zenodo.17613617

Abstract

Model-Based Systems Engineering (MBSE) has advanced Systems Engineering by establishing models as the primary asset, but its reliance on centralized tools and limited integration with other engineering data restricts its industrial impact, particularly in federated environments and interdisciplinary use cases. To address these limitations, this paper introduces Data-Centric Systems Engineering (DCSE), an approach that places semantic data, rather than models or tools, at the core of Systems Engineering. DCSE is realized through a Semantic Digital Thread (SeDT), a knowledge graph built on a five-layer Semantic Data Model (SeDM) that semantically integrates artefacts from structured data sources, unstructured data sources, and engineering models such as 3D-CAD and Modelica models. Furthermore, this paper outlines the conceptual principles of DCSE and describes methods for constructing and governing the SeDT. Using two representative use cases, “Change Impact Analysis” and “Change Cost Estimation”, it demonstrates how DCSE enables traceable, and explainable decision-making across disciplines and lifecycle stages.

Keywords: Model-Based Systems Engineering (MBSE); Product Lifecycle Management (PLM); Data-Centric Systems Engineering (DCSE); Semantic Digital Thread (SeDT)

1. Introduction

Systems Engineering is a multidisciplinary discipline that focuses on designing, developing, and managing complex systems across their entire lifecycle. It encompasses activities from conceptual design and requirements management to verification, validation, and operation. SE provides a structured process for aligning technical specifications with stakeholder needs and business objectives, ensuring that systems are fit for purpose [1]. Traditional SE practices have relied heavily on document-based processes, where knowledge is distributed across textual artifacts such as specifications, spreadsheets, and reports. These often lead to inconsistency, limited traceability, and loss of knowledge across lifecycle phases. MBSE was developed to address these challenges by making models the primary resource in SE [2]. However, in industrial practice, MBSE often falls short when engineering data is distributed across heterogeneous tools and disciplines. Furthermore, semantic consistency across lifecycle phases cannot be guaranteed, and AI and analytics cannot reliably consume the available data due to a lack of explicit semantics. These limitations motivate a more data-centric architectural approach.

1.1. The Current State and Vision of MBSE

INCOSE defines MBSE as *"the formalized application of modeling to support system requirements, design, analysis, verification and validation activities beginning in the conceptual design phase and continuing throughout development and later life cycle phases. MBSE is part of a long-term trend toward model-centric approaches adopted by other engineering disciplines, including mechanical, electrical and software"* [3].

Although the objective of MBSE is to model a system across the full system lifecycle, most MBSE approaches only support a few technical processes, such as the system requirements definition process, the architecture definition process and the design definition process, as well as parts of the verification and validation processes. These processes are only found in the early stages of SE, specifically the concept stage and, to a certain extent, the development stage [2].

INCOSE has outlined a comprehensive vision for the evolution of Systems Engineering, describing both the current state of practice and the aspirational future state envisioned for 2035. Currently, *"Advancing digital technologies and standards development are enabling Model-Based Systems Engineering (MBSE) practices, but modeling ecosystems are often rudimentary and incomplete. Additionally, computation, cloud infrastructure, and data discovery are not being fully leveraged compared to other engineering and scientific disciplines."* [4]. This characterization clearly highlights the limitations of today's SE and

MBSE practices, where digital technologies are only partially leveraged and data remain fragmented across tools and domains.

INCOSE contrasts this with a forward-looking vision of what Systems Engineering could become in the next decade. According to INCOSE in 2035, *“Systems Engineering environments fully leverage advances in digital technologies and modeling standards to enable rapid exploration of designs using high fidelity simulation, data visualization, and semantic web technologies. Systems Engineering tools benefit from internet connectivity and knowledge representation to provide seamless exchange of information with other disciplines and their tool environments as part of a broader enterprise digital engineering environment. Further, systems engineers partner with machines to combine creativity and automation in a robust and agile design process.”* [4].

Similar concerns about the limitations of application-centric thinking have been articulated in the business and IT domains. McComb argues that the prevailing focus on monolithic applications and tightly coupled data models leads to fragmented, inflexible enterprise landscapes and significant “software wasteland” [5], [6]. He advocates for data-centric architectures in which shared, explicit semantic models outlive individual applications.

1.2. Related Works

Several researchers have contributed to advancing the integration of model-based approaches and semantic technologies in Systems Engineering, providing valuable building blocks toward the digital engineering environment envisioned by INCOSE for 2035.

Pfenning introduces the concept of Anchor Elements in his doctoral work [7] as a means of addressing the challenge of interdisciplinary collaboration across engineering disciplines. He defines a common subset of digital artefacts, such as requirements, CAD models, parameters, part lists or test protocols, to enable interdisciplinary communication. Rather than prescribing a fixed taxonomy, the Anchor Element concept focuses on identifying the essential artefacts for cross-disciplinary collaboration. This abstraction is operationalised through the Kaiserslautern System Concretion Model (KSCM), which structures system development into multiple 'spaces': the requirements space, the solution space, the verification and validation space, and the administration space. Within the solution space, four abstraction layers (contextual, functional, logical and physical) allow system elements to be linked through defined relations, such as 'satisfy', 'derive', 'allocate' and 'verify', which enable traceability across layers.

However, Pfenning’s work primarily focuses on linking anchor elements across tools and abstraction layers without integrating system and organizational data semantically. While the proposed framework captures the relationships between configuration items and

engineering models (e.g., CAD, 1D-Simulation), it does not yet establish a unified semantic model that encompasses all semantic levels, from the enterprise upper ontology to the persistence level. Furthermore, the approach does not involve the automated extraction of data from engineering models, nor does it consider unstructured data sources, such as textual requirements or documents. Consequently, although the Anchor Element and KSCM concepts provide a pragmatic basis for linking configuration items and engineering models to facilitate interdisciplinary collaboration, they remain centered on structural integration rather than the comprehensive data-centric integration of engineering data required for a Semantic Digital Thread.

The authors of [2] and [8] proposed a *Holistic Model-Based Engineering (HMBE)* approach for industrial plant engineering and construction. In two complementary papers, the authors present a pragmatic strategy for introducing MBSE into organizations by enabling the (semi-)automatic creation of system models using extracted data from domain-specific models, such as Modelica or 3D-CAD. The approach emphasizes continued use of existing engineering tools, ensuring that domain experts can work within familiar environments while contributing to a unified system model. Through ontology-based relationships, the method enhances traceability and supports impact analysis of design changes. Despite these advances, the proposed framework supports structural traceability but does not yet provide full semantic integration of engineering knowledge across disciplines. Furthermore, automated knowledge discovery and automatic cross-domain semantic linking are lacking.

The studies [9] and [10] have made contributions toward semantic integration in MBSE through two complementary studies. In [9], a semantic MBSE approach for metal forming process design is proposed. Using the multi-architecture modeling language KARMA, the authors define the metal forming process across mission, operational, functional, logical, and physical perspectives. The KARMA models are automatically transformed into ontology models, which are then converted into Petri Net (PN) models for dynamic simulation and early design verification. In [10], the same semantic MBSE framework is applied to the design of an aircraft Prognostics and Health Management (PHM) system. While these studies leverage selected aspects of Semantic Web technology for model transformation and reasoning, they remain primarily domain specific. Consequently, they lack focus on enterprise-scale data interoperability and cross-disciplinary semantic integration. The generated ontologies are not suitable to serve as a semantic model covering heterogeneous engineering tools and lifecycle stages.

In summary, existing approaches either focus on structural integration across tools, domain-specific semantic modeling, or partial ontology-based traceability. None of them provide an enterprise-scale, layered semantic data model that spans from upper ontologies

to persistence schemas, and an operational pattern that integrates deterministic graph analysis with guarded LLM-based synthesis for decision support. DCSE and SeDT address this gap.

1.3. Author’s Proposal

In contrast to existing approaches, DCSE and SeDT aim to provide a general, enterprise-scale semantic backbone that unifies structured, unstructured, and engineering-model data, designed explicitly for federated heterogeneous IT environments and AI-supported decision-making.

The INCOSE envisioned 2035 aligns closely with the objectives of “Data-Centric Systems Engineering (DCSE) based on the Semantic Digital Thread (SeDT)” that is proposed in this paper. By employing semantic web technologies, DCSE directly addresses the gaps identified in the current state, enabling data interoperability, semantic reasoning, and seamless integration across the engineering disciplines. In doing so, it provides a concrete pathway toward realizing INCOSE’s 2035 vision for a fully connected, intelligent, and collaborative Systems Engineering environment. The SeDT serves as the DCSE’s backbone, providing the vital semantic data required for DCSE’s use-cases. This paper makes the following contributions:

- It introduces **Data-Centric Systems Engineering (DCSE)** as an evolution of MBSE, positioning semantic data as the primary asset and clarifying its differentiation from traditional MBSE approaches.
- It defines a **Semantic Digital Thread (SeDT)** underpinned by a five-layer **Semantic Data Model (SeDM)** that spans from enterprise upper ontologies to persistence models. The SeDT enables semantic integration of structured, unstructured, and engineering model-embedded data in a deterministic knowledge graph.
- It proposes a technology-agnostic **operational pattern** for DCSE based on hybrid retrieval and guarded LLM synthesis, demonstrating its application to change impact analysis and cost estimation in an automotive company.

The remainder of this paper is structured as follows. Section 2 introduces the Semantic Digital Thread (SeDT) and its underlying Semantic Data Model (SeDM). Section 3 defines Data-Centric Systems Engineering (DCSE), positions it relative to MBSE, and illustrates its application to change impact analysis and change cost estimation. Section 4 summarizes the key findings and outlines directions for future work.

1.4. Acronyms and Notation

Term	Meaning	Term	Meaning
API	Application Programming Interface	RDF	Resource Description Framework
CAD	Computer-Aided Design (e.g., 3D-CAD)	SE	Systems Engineering
DCSE	Data-Centric Systems Engineering	SeDM	Semantic Data Model
DDD	Domain-Driven Design	SeDT	Semantic Digital Thread
ECAD	Electrical Computer-Aided Design	SHACL	Shapes Constraint Language
MBSE	Model-Based Systems Engineering	SKOS	Simple Knowledge Organization System
PLM	Product Lifecycle Management	OWL	Web Ontology Language

2. Semantic Digital Thread (SeDT)

The Semantic Digital Thread (SeDT) represents the backbone of Data-Centric Systems Engineering (DCSE). It uses a unified, machine-interpretable Semantic Data Model (SeDM) as a blue print to connect engineering data across disciplines and lifecycle stages. The SeDM defines the semantic schema of entities, relationships, and constraints, while the SeDT is the instantiated knowledge graph that materializes these definitions using data from enterprise sources. Unlike traditional digital threads, which focus on linking data through technical identifiers, the SeDT establishes semantic relationships between system artefacts, creating a context-aware graph.

At its core, the SeDT is a knowledge graph that unites data from structured and unstructured sources, as well as data concealed within Engineering models such as 3D CAD, Modelica, ECAD, and Simulink models. The graph is updated continuously with new data, enabling a near-real-time reflection of the system's state and its evolving configuration. This chapter introduces the SeDM and the creation of the SeDT.

2.1. Semantic Data Model (SeDM)

The Semantic Data Model (SeDM) forms the foundation upon which the SeDT is built. Built around a five-layered structure, the SeDM establishes a unified semantic foundation that spans from abstract enterprise concepts to concrete database schemas. Its layered approach clearly separates conceptual reasoning from physical implementation, ensuring that business semantics remain traceable across every abstraction level from enterprise-wide ontologies to operational databases.

The SeDM comprises five layers: Enterprise Upper Ontology, Integration Model, Terminology Model, Domain Models, and Persistence Models. Figure 1 illustrates these layers and gives examples of each one.



Figure 1: Semantic Data Model (SeDM) layers.

2.1.1. Enterprise Upper Ontology

At the core of the SeDM lies the Enterprise Upper Ontology, which defines the high-level, domain-agnostic classes. These classes form the universal conceptual vocabulary from which all downstream models derive. Organized into categories such as physical entities, software, processes and activities, requirements, organizations, and spatial or temporal abstractions, this ontology provides a consistent backbone for representing every aspect of enterprise knowledge. Through properties such as identification, temporal markers, and status indicators, the Enterprise Upper Ontology ensures that all entities maintain a consistent metadata model.

Enterprise Upper Ontology evolves iteratively as organizational needs and domains mature, and can incorporate standardized ontologies. It supports extensibility, allowing all

specialized domain concepts in lower layers to inherit from a common foundation, thus promoting coherence across the enterprise.

2.1.2. Integration Model

The Integration Model forms the second layer, defining predicates (object properties) that describe the relationships between entities. These relationships cover concepts such as part-whole hierarchies, requirement traceability, process execution, control dependencies and ownership. They form the structural and semantic glue that connects all entities across domains. The Integration Model extends beyond engineering to encompass the entire product lifecycle, as well as operational and organizational aspects. The Figure 2 illustrates a selection of entities from the Enterprise Upper Ontology and Integration Model.

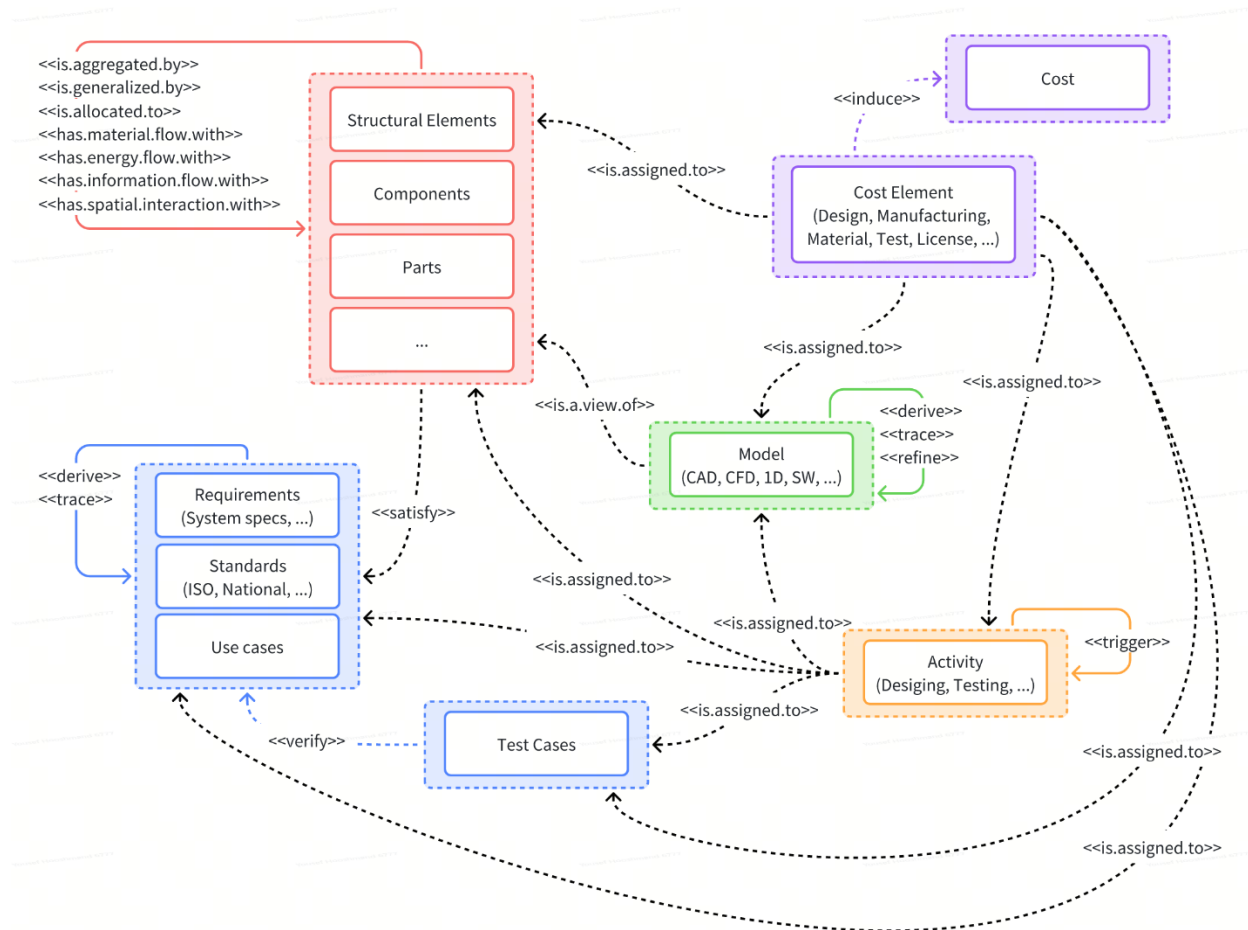


Figure 2: Semantic Data Model – Sample entities of Enterprise Upper Ontology and Integration Model. Adopted from [8] and [11].

Semantic relationships drastically increase traceability and show how high-level stakeholder requirements are translated into implementable artefacts that are validated

through concrete test cases. For example, key relationships such as 'isPartOf', 'satisfy', and 'verify' establish comprehensive linkages between business and engineering artefacts. Through these, it is possible to trace, in a single semantic chain, how a specific test scenario verifies a requirement that a component satisfies. Symmetric and inverse properties reinforce bidirectional navigation, enabling advanced semantic querying. The Integration Model is also mainly domain-agnostic and serves as universal semantic middleware, ensuring that all enterprise artefacts can be linked using a coherent relational grammar.

2.1.3. Terminology Model

The Terminology Model operationalizes semantics by using industry- and company specific vocabulary. Systematically derived from the Enterprise Upper Ontology, it contains terms that formalize the language actually used in different disciplines. This layer bridges the gap between academic ontology and practical lexicons by incorporating terminologies from multiple products, systems and organizational domains. These are traditionally often maintained in terminology services or Master Data Management (MDM) solutions.

SKOS¹ (Simple Knowledge Organisation System) standards are employed for alternative and preferred labels, enabling the management of synonyms (e.g. 'EV' and 'Battery Electric Vehicle'). Besides company-specific terms, the Terminology Model can incorporate a standardized vocabulary, such as ECLASS². This level of semantic precision ensures an unambiguous understanding of terms in structured and unstructured data sources, enables efficient communication across disciplines, and guarantees interoperability based on machine-readable terminologies.

2.1.4. Domain Models

Building upon previous layers, Domain Models define the fourth layer, where abstract ontological constructs are instantiated within specific contexts. The architecture supports both system domains (e.g., Lighting, Battery, Seating) and engineering or organizational domains (e.g., Configuration Management, Quality, Manufacturing). While the domain models are primarily generic, project-specific domain models can also be created and added to this layer. Each domain model encapsulates domain-specific entities, business logics, rules, and relationships.

The system-specific domain models define the components, requirements and other entities of the system. These will then be linked using relationships defined in the integration model or domain-specific relationships. For example, in the Lighting System Domain, the

¹ <https://www.w3.org/TR/skos-reference/>

² <https://eclass.eu/en/>

HeadlampAssembly is composed of components like LED, LensUnit, and HeatSink. Requirements such as LuminousIntensity or Thermal are explicitly linked to specific components and validated through corresponding tests. The responsibility for building and validating system-specific domain models lies with domain experts.

Organizational domain models are primarily company-specific and should be defined based on business capabilities, in line with the Domain-Driven Design (DDD) approach [12] and ideally the guidelines set out in [13]. This approach is well suited to manufacturing organizations with a heterogeneous PLM landscape and engineering toolchain.

2.1.5. Persistence Models

The final layer, Persistence Models, links the previous semantic definitions to physical persistence structures. Each entity (tables, columns, entities, graphs) explicitly traces back to corresponding semantic elements through integration mappings, ensuring bidirectional traceability between physical data and its conceptual definition. For example, the ComponentCatalog table in the BOM Platform database maps directly to the Component class in the ontology. This layer also covers the definition and metadata of API interfaces needed to retrieve data from applications.

In addition to the obvious benefits of interoperability and traceability, this layer enables integrated analytics, such as requirement-to-test coverage, supplier performance evaluation and cost roll-ups. This is achieved by combining data from multiple systems via shared semantics. The result is a unified data landscape that supports data-centric use cases and cross-disciplinary collaboration. The database model ensures full semantic lineage for each entity.

2.2. Building the Semantic Foundation

The Semantic Digital Thread (SeDT) is the operational backbone of the DCSE. It enables the dynamic, semantically consistent integration of heterogeneous enterprise data sources into a unified, provenance-rich knowledge graph. This process combines deterministic extraction pipelines with AI-assisted semantic interpretation and human-in-the-loop curation. The consistency of the SeDT is ensured by enforcing conformance to the SeDM through machine-checkable constraints.

Figure 3 illustrates the SeDT generation pipeline, highlighting how structured, unstructured, and model-embedded data are transformed via intermediate representations into a governed knowledge graph.

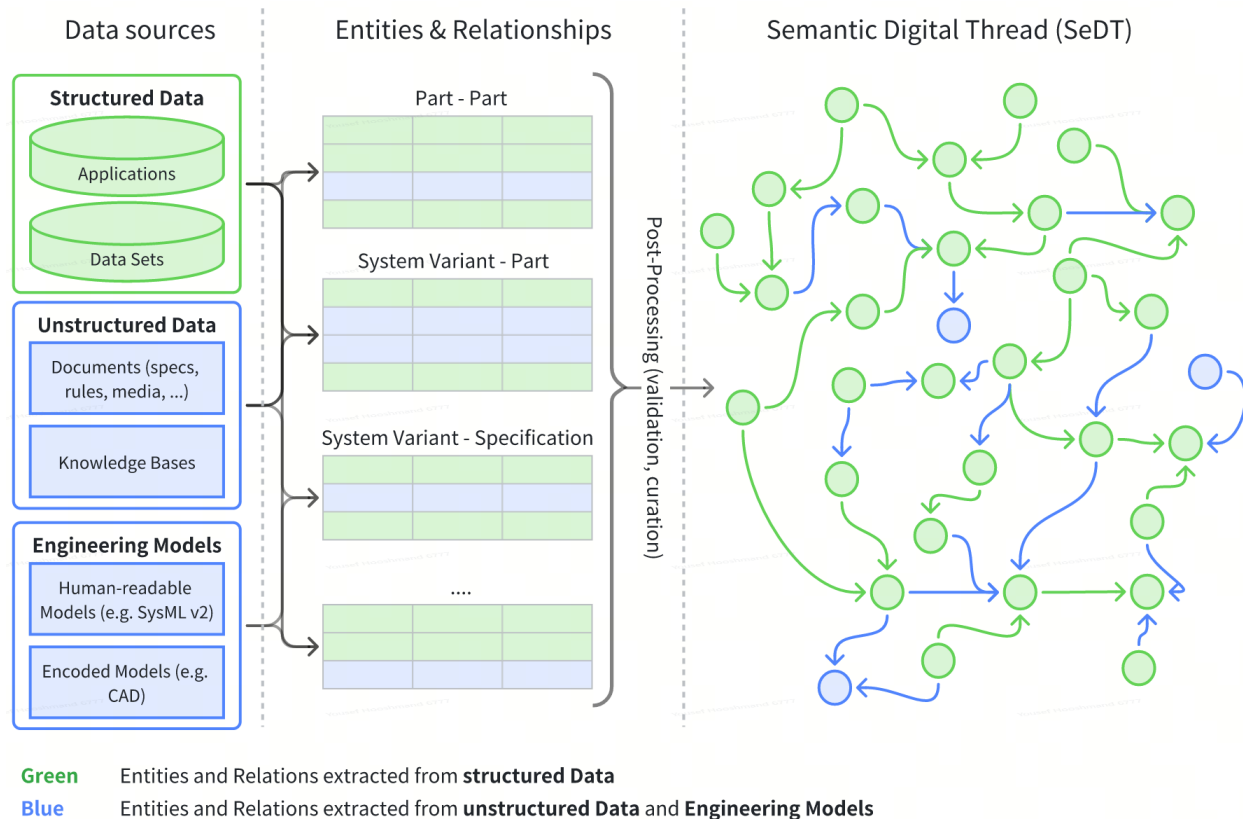


Figure 3: Building the Semantic Digital Thread (SeDT).

2.2.1. Data Extraction

Data extraction strategies vary considerably depending on the data modality as the primary aspect, as well as secondary aspects such as data volume, data quality, required update frequency, and transformation complexity. For data governance and quality assurance reasons, the definitive creation and insertion of internal company-specific entities (e.g., Part P1, Requirement R1) into the knowledge graph are restricted to verified, structured sources. Unstructured sources are primarily used to extract relationships between existing entities or to identify external, contextual entities, as the identity and validity of internal entities are asserted deterministically in the source systems.

Structured Data: For this, schema-aware pipelines map source structures directly to SeDM classes and predicates. Primary and foreign keys are converted into semantic identifiers and relations, ensuring deterministic traceability.

Unstructured Data: Here, domain-adapted natural language processing (NLP) pipelines perform named entity recognition, relation extraction, and coreference resolution. These pipelines are enhanced with the use of large language models (LLMs) to generate context-sensitive candidates, which are then subject to automatic validation and curator review.

Terminology grounding via the SeDM ensures semantic alignment and mitigates the risk of linguistic drift or hallucination.

Engineering Models: In the context of engineering models, semantic extraction focuses on key parameters and metadata embedded in engineering models, such as 3D-CAD, ECAD and Modelica models. These parameters are then parsed and instantiated as SeDM domain model entities. This establishes an explicit link from the parameters in engineering models to other engineering artefacts, such as verification and test items. Custom, model-specific data-extraction solutions are required to extract parameters from engineering models. Therefore, only models approved by domain experts will be added to the data extraction pipeline. Furthermore, the required lifecycle state of the model, such as 'frozen' or 'released', will be decided on a case-by-case basis.

In order to achieve operational scalability, workloads must be partitioned according to data velocity and payload size. High-frequency updates are processed via low-latency streaming lanes. In contrast, complex artefacts such as CAD models should be handled in batch-oriented pipelines.

2.2.2. Intermediate Representation and Transient Persistence

The explicit use of an Intermediate Representation (IR) layer decouples heterogeneous, noisy inputs from the canonical SeDT. Each IR record retains candidate semantic mappings, raw source fragments, and confidence metadata, ensuring transparent and traceable transformations. This approach supports scalable parallel processing, facilitates reprocessing of failed batches, and provides a clear audit trail. Depending on throughput requirements, IR data may be stored temporarily in memory or persisted in appropriate repositories, such as document-oriented databases.

Human-in-the-loop curation remains integral across all modalities. Domain engineers, ontology stewards and data curators oversee low-confidence mappings, propose ontology extensions and resolve conflicting data. These interventions are logged as first-class provenance events and feed back into automated extraction heuristics, thereby improving the long-term robustness of the model.

2.2.3. Incremental Update Strategy

The SeDT is designed for incremental regeneration, maintaining synchronization with evolving enterprise data. Each modification to a source system triggers the emission of a delta event representing creation, update or deletion, which in turn triggers semantic differencing and localized recomputation. Deterministic mechanisms based on stable semantic identifiers and source data ensure idempotency and prevent data duplication.

Temporal semantics are explicitly modelled through validity intervals and configuration tags, enabling the retrospective reconstruction of the system's states.

2.2.4. Governance, Quality, and Compliance

Long-term trust in SeDT depends on robust governance mechanisms. Automatic validation, for example based on SHACL³, ensures full SeDM compliance, for instance by enforcing that requirements are only linked to design elements that satisfies them, and that test cases verifying a requirement include measurements consistent with its defined physical units. Every entity in the graph carries provenance annotations, and curator interventions are logged with reviewer identity and rationale. Ontology and terminology updates follow controlled proposal–review–publication cycles, with regression testing to ensure backward compatibility and model stability.

2.3. The Essence of SeDT

In essence, SeDT turns enterprise data into a context-aware, machine-interpretable fabric where requirements, designs, tests, processes, and organizations are linked through coherent semantics according to SeDM. Thus, the definition and its practical test are both straightforward:

“You only have a true Semantic Digital Thread if anyone in the organization is only one query away from finding, understanding, and trusting any relevant data.”

The SeDT ingests only a targeted subset of data from source applications to construct the knowledge graph, rather than replicating entire datasets. When a user interaction requires information beyond what is stored in the graph, such as additional attributes, 2D drawings, 3D CAD models, or detailed transactional data, the SeDT dynamically calls the corresponding source system APIs to retrieve and enrich the response with the necessary data in real time while taking access permissions into account.

3. Data-Centric Systems Engineering (DCSE)

This chapter introduces Data-Centric Systems Engineering (DCSE) as an evolution of Model-Based Systems Engineering (MBSE), placing semantic data at the centre of Systems Engineering rather than models or tools. It defines DCSE, outlines its scope and distinguishing characteristics, and explains how the Semantic Digital Thread (SeDT) can enable high-value, cross-domain use cases, such as change impact analysis and change cost estimation. The chapter concludes with a pragmatic operational pattern combining

³ <https://www.w3.org/TR/shacl/>

deterministic graph analysis with context-based retrieval and guarded Large Language Model (LLM) synthesis to support informed decision-making.

3.1. DCSE: Definition, Scope, and Differentiation

DCSE Definition: “Data-Centric Systems Engineering (DCSE) is an approach that places data and its interconnections as a primary asset at the core of Systems Engineering, ensuring traceability across disciplines, lifecycle phases, and software tools.”

Instead of organizing engineering artefacts around specific tools or modeling languages, DCSE organizes them around a unified Semantic Data Model (SeDM), realized with Semantic Web standards such as RDF⁴ and OWL⁵. In doing so, DCSE addresses several limitations of current MBSE practice. These include the tight coupling to centralized modeling tools, limited interoperability in federated environments, and the difficulty of integrating structured and unstructured artefacts at scale. The key aspects of DCSE and how it differs from MBSE are listed in Table 1 and summarized below.

In contrast to traditional MBSE, which often centers on the system model within a specific modeling environment, DCSE broadens the concept to encompass all semantically represented artefacts and their interrelations. In this sense, DCSE can be viewed as a superset of MBSE that is deliberately designed for interoperability, extensibility, and AI readiness. Accordingly, MBSE becomes indeed one important contributor to, and consumer of, the SeDT.

Table 1: The comparison of DCSE and MBSE

Data-Centric Systems Engineering (DCSE)	Model-Based Systems Engineering (MBSE)
Data as the central, enduring asset	Model as the central, enduring asset
Application- agnostic by design	Application- driven by design
Inherent semantic interoperability	Technical interoperability
Efficient in federated IT environments	Efficient in centralized environments
Based on Semantic Web Stack	Typically based on SysML-v2 (KerML) or ARCADIA
Machine-interpretable explicit semantics	Model semantics within a modeling language
Federated governance & consistency assurance	Central governance & consistency assurance

Data as the central, enduring asset: In DCSE, the Semantic Data Model (SeDM) and the data in Semantic Digital Thread (SeDT) outlive specific file formats, and applications.

⁴ <https://www.w3.org/RDF/>

⁵ <https://www.w3.org/OWL/>

Engineering tools are treated as transient means to create, manipulate, and visualize data, not as the primary value carrier. This perspective supports long-term lifecycle concerns, such as reuse of knowledge across product generations, regulatory traceability, and robust configuration management, because the authoritative reference is the semantic data itself.

Application-agnostic by design: By decoupling the data layer from individual tools, DCSE explicitly avoids vendor lock-in and monolithic architectures. The Semantic Digital Thread (SeDT) acts as a shared integration backbone that can be accessed from a best-of-breed toolchain. Mechanical, electrical, software, and systems engineers can continue to use their specialized tools, while contributing to and retrieving from the same authoritative data source. This is particularly important in industrial contexts where tool ecosystems are heterogeneous and evolve over time.

Inherent semantic interoperability: DCSE employs semantic web standards such as RDF, and OWL to represent entities, relationships, and constraints in a machine-interpretable form. This goes beyond syntactic data exchange formats, enabling applications to understand, and exchange data consistently based on shared meaning. Consequently, automated validation, and analysis across tools and domains become feasible. For instance, a “brake caliper” represented in CAD, in a Modelica model, and in a requirement specification can be recognized as instances of the same conceptual element in SeDT, enabling consistent traceability and analysis.

Efficiency in federated IT environments: Many industrial organizations operate in federated IT landscapes, where different business units, suppliers, and partners manage their own tools and infrastructures. DCSE provides an architectural pattern that connects these distributed environments via a Semantic Digital Thread (SeDT), rather than forcing data into a single centralized repository or application. Data can remain close to its source, yet still be integrated semantically through mappings and alignment to the common data model.

Machine-interpretable explicit semantics: The Semantic Data Model (SeDM) and the data in Semantic Digital Thread (SeDT) are high-quality, context-rich foundations that enable the efficient implementation of DCSE use-cases. As entities and relations are explicitly typed and linked, algorithms can exploit structural and semantic information instead of relying on isolated documents or models. This is particularly valuable for cross-domain tasks such as change impact analysis, change cost estimation, anomaly detection, or design space exploration, where relevant information is distributed across multiple isolated tools and data sources. Furthermore, semantic content is essential for any AI solution.

Federated governance and consistency assurance: While DCSE does not prescribe centralized control over tools and processes, it does emphasize the governance of the unified semantic model. Federated governance mechanisms ensure that concepts, relationships, and constraints are defined, curated, and evolved in a controlled manner. Consistency between artefacts originating from different applications is maintained through semantic mappings, integrity constraints, and quality checks on the SeDT. This approach enables local autonomy at the tool and domain level while ensuring global coherence at the semantic data level.

3.2. Use Cases: Change Impact Analysis and Cost Estimation

The value of DCSE becomes most apparent when answering complex, cross-domain questions that require data from different applications. Change impact analysis and change cost estimation are representative examples. In conventional, tool-centric environments, these analyses are labor-intensive, dependent on individual experts' tacit knowledge, and difficult to audit. SeDT as the foundation of DCSE provides the semantic mesh needed to make such analyses systematic and repeatable. This paper uses the change of brake caliper material as a representative use case in an automotive context.

3.2.1. Use Case: Changing Brake Caliper Material

Assume a design decision to switch the brake caliper material from aluminum to a composite, primarily to reduce vehicle weight. Even this apparently local change has wide-ranging implications (see Figure 4).

Direct technical impacts: The material change affects brake system behavior, caliper mounting interfaces, thermal performance, and durability. It may require adjustments to control software parameters (e.g., for brake force distribution or stability control), and recalibration of embedded systems. In SeDT, these relationships are captured through explicit links such as `isAggregatedBy`, and `hasMaterialFlowWith`.

Indirect and systemic impacts: Beyond the immediate subsystem, the change can influence suspension dynamics, steering feel, Advanced Driver Assistance Systems (ADAS) performance, noise-vibration-harshness (NVH) characteristics, manufacturability, and maintainability. These second-order effects are often overlooked in siloed analyses. In SeDT, a deterministic impact subgraph can be constructed that covers both direct and indirect neighbors.

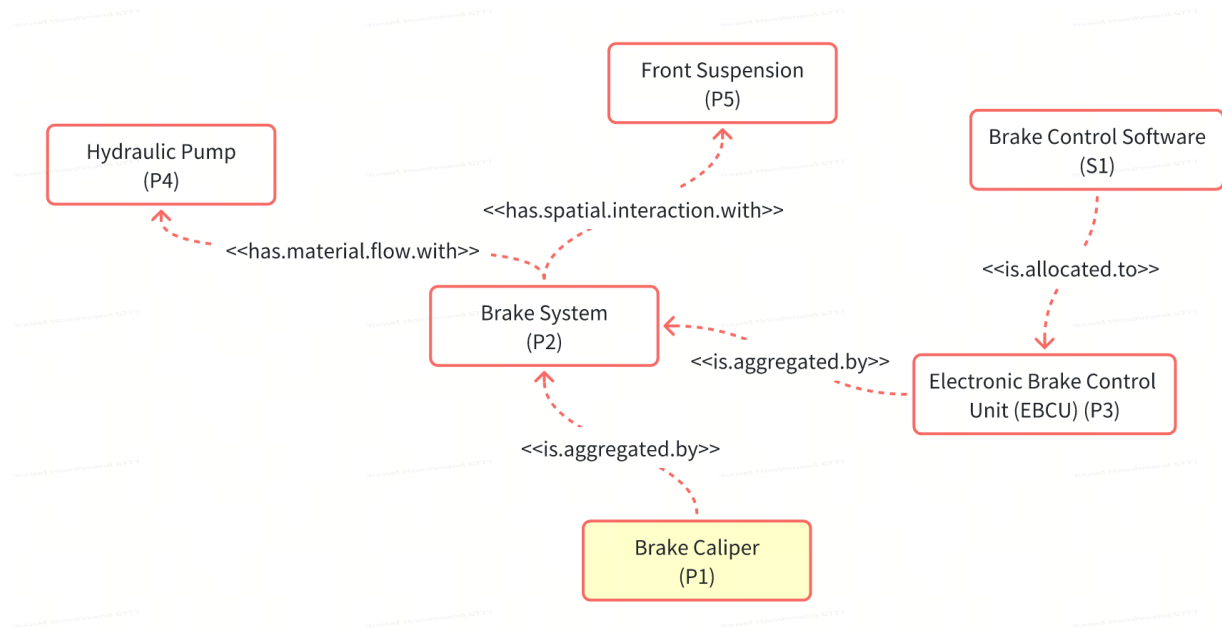


Figure 4: Use case “Changing Brake Caliper Material”.

In addition to direct and indirect neighboring components, entities such as requirements, specifications, engineering models and test cases also need to be covered to gain a holistic view. The SeDT makes these dependencies explicit by linking them to the system components (see Figure 5).

Requirements: Technical specifications, legal and safety constraints, and user needs are modeled as requirement entities, which are linked to other entities, such as components, simulation models, and verification plans. These entities can be either atomic requirements or full specification documents (e.g. PDF files).

Design and simulation artefacts: Parameters and structures from 3D-CAD, Modelica, or other engineering models are extracted and mapped to the SeDT, so that geometric, material, and behavioral aspects of the components can be queried uniformly. In addition to the model parameters, the models themselves are referenced as entities in SeDT.

Testing and validation artefacts: Test specifications, test rigs, simulation configurations, and test results are represented as entities that are linked to the requirements and system elements that they verify. A material change may trigger the creation of new test cases, the modification of benches, or the updating of simulation models.

Hidden and unstructured knowledge: Engineering knowledge is often embedded in natural language documents (specifications, change requests, design reviews, field reports, internal knowledge bases) or resides as tacit knowledge in experts’ heads. DCSE grounds unstructured artefacts in the SeDM, enabling the extraction of entities and relations. These

will be then integrated into the SeDT after validation and, potentially, curation. As a result, both explicit and implicit entities and relationships become navigable within a single semantic fabric.

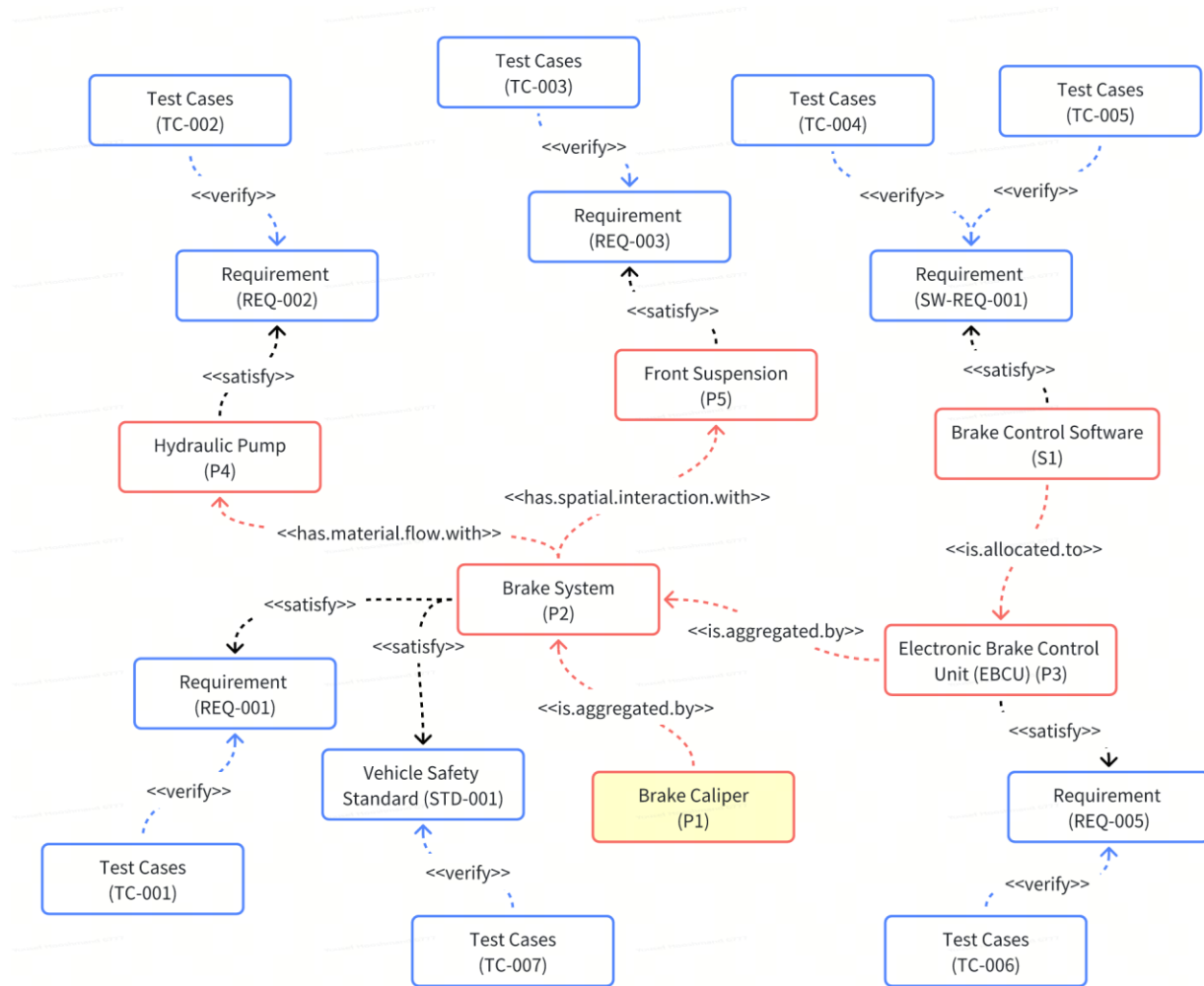


Figure 5: Use case - Impact graph extended with engineering artefacts.

3.2.2. Induced and Secondary Costs

Every change has a financial impact, which may be due to direct or indirect costs. Therefore, in order to estimate the cost of the change, the SeDT needs to provide data on cost elements that are relevant to the entities affected (see Figure 6).

Direct costs: These include, for example, material price differences, process changes (e.g., new machining steps), modifications to fixtures and tooling, and supplier onboarding or qualification. In SeDT, such cost entities and drivers are modeled explicitly and associated with the affected components, processes, and suppliers.

Indirect costs: These encompass additional testing effort, documentation updates, simulation/model rework, and re-certification activities, for example. These are often difficult to quantify in traditional setups because the relevant dependencies are not systematically captured.

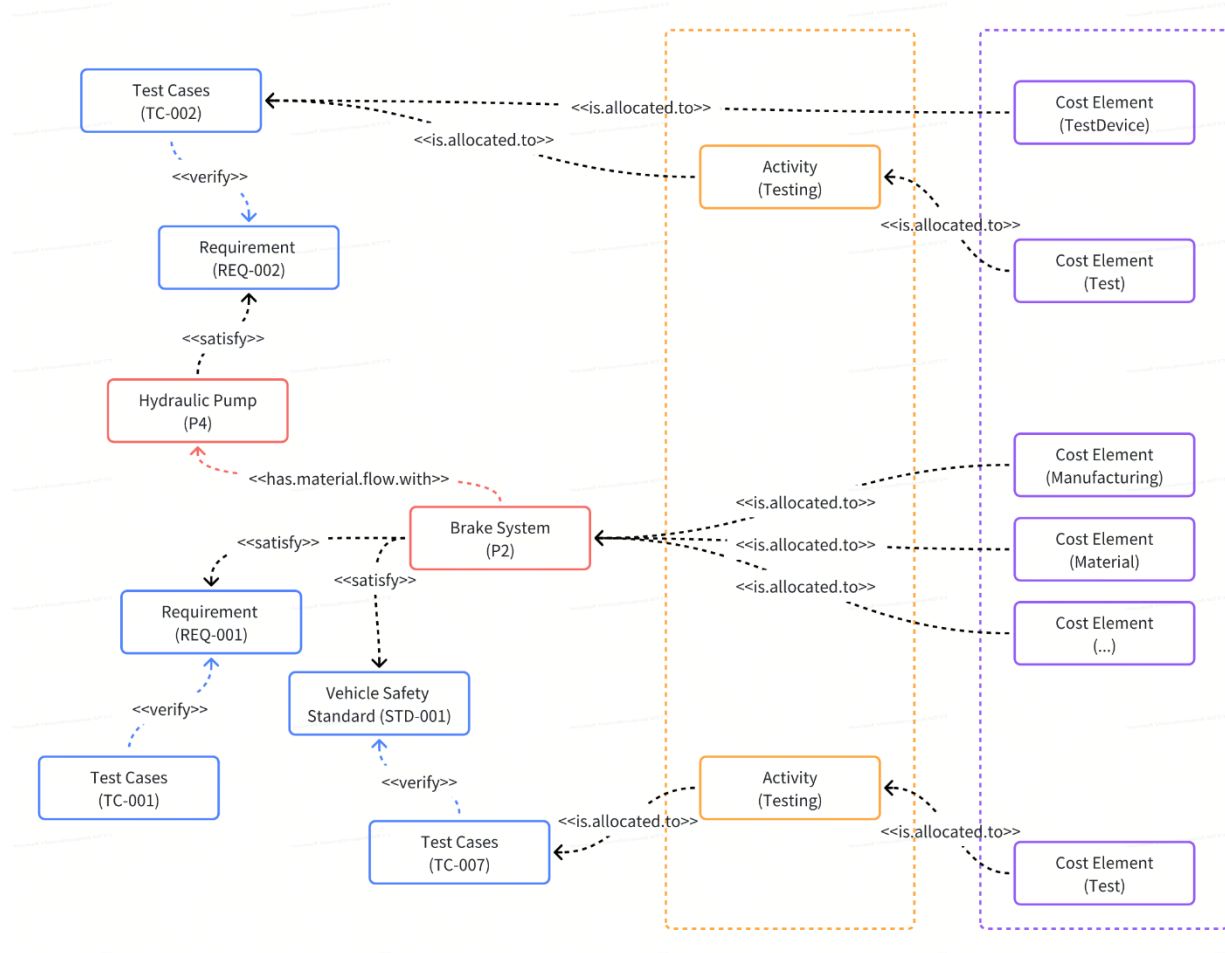


Figure 6: Use case - Impact graph extended with cost elements.

Within DCSE, cost-relevant entities are linked to affected artefacts across disciplines. This enables the creation of traceable and explainable roll-ups of cost estimates: every aggregated cost figure can be broken down to its constituent parts and the artefacts that necessitate them.

Overall, it is difficult to perform reliable change impact analyses and cost estimations using current siloed solutions. The next section will explain the procedure for such a use case based on SeDT.

3.3. Usage Pattern: Hybrid Retrieval and Decision Support

To operationalize DCSE in day-to-day work, a hybrid retrieval and decision support pattern is required that combines deterministic graph querying on the SeDT with context-based retrieval from unstructured sources and guarded LLM-based synthesis. This pattern is demonstrated here using the brake caliper material change as an example, but it is applicable to a wide range of cross-domain use cases (see Figure 7).

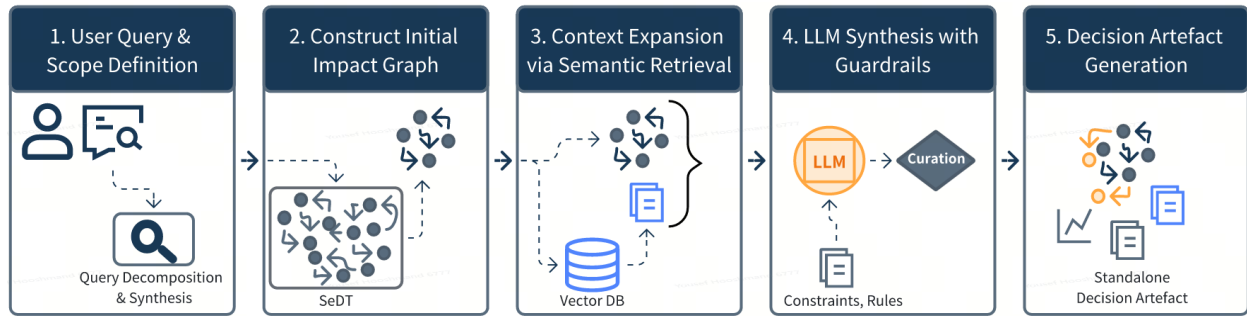


Figure 7: Hybrid retrieval and decision support steps.

Step 1: User Query and Scope Definition: An engineer formulates a question, such as: “What is the impact and estimated cost of changing the brake caliper material to composite?” The system parses the questions, identifies relevant entities, relations, and constraints, and anchors them to concepts in the SeDM. Optionally, the user can refine the scope (e.g., specific vehicle variants, lifecycle phase, regulatory context, or part numbers). Finally, a graph query will be created.

Step 2: Construct Initial Impact Graph: Based on the anchored entities and relations, graph queries, for example based on SPARQL⁶, will be executed against the SeDT to construct an initial impact graph. This graph includes:

- direct and indirect affected components and assemblies,
- linked requirements and associated verification artefacts,
- engineering models and parameters extracted from them,
- relevant suppliers, manufacturing steps, and process parameters,
- explicitly modeled cost elements associated with these artefacts.

The constructed impact graph is a deterministic subgraph grounded in curated semantic data, serving as a high-precision baseline for subsequent analysis.

Step 3: Context Expansion via Semantic Retrieval: To capture additional, possibly implicit information, the system retrieves further content from linked unstructured data such as

⁶ <https://www.w3.org/TR/sparql11-query/>

requirement texts, specifications, and incident reports. A vector database, grounded in the Terminology Model, is used to perform semantic similarity search. This step enriches the context around the initial impact graph without compromising its deterministic structural integrity.

Step 4: LLM Synthesis with Guardrails: A Large Language Model is then used to synthesize a response. To ensure reliability and alignment with engineering governance, the LLM operates under explicit guardrails:

- Its input is constrained to the initial impact graph, the retrieved passages, applicable semantic models and constraints, and governance rules (e.g., domain-specific templates).
- The LLM is responsible for tasks such as proposing extended impacts on entities in SeDT (e.g. components, legal requirements), identifying potential secondary effects, and outlining direct and indirect cost implications.
- The LLM is not allowed to introduce new facts into the SeDT directly. Any newly identified entities, relationships or cost drivers need to undergo automatic validation, such as consistency checks, and also curator review. Thus, before the initial impact graph is enriched, human experts can review, approve or reject these proposed assertions.

This guarded synthesis leverages the generative capabilities of LLMs while maintaining the trustworthiness and auditability required in engineering contexts.

Step 5: Decision Artefact Generation: The outcome of the final step is a comprehensive decision artefact that can be stored, shared, and revisited. It can be used standalone and typically includes:

- a visual and queryable representation of the final impact graph,
- a rationale explaining the identified impacts with explicit citations to graph nodes and document fragments,
- a structured cost breakdown differentiating direct and indirect costs, with traceability to underlying drivers,
- recommended actions, such as additional components to consider, tests to perform, documents to update, and stakeholders to involve.

As the decision artefact is persisted, subsequent changes, validations or decisions can reference it. This enables a longitudinal analysis of how decisions were made and their effect on system evolution. In addition, the relevant entities and relationships identified during the impact analysis task can be added to the SeDT upon expert approval.

3.4. The Essence of DCSE

At its core, Data-Centric Systems Engineering (DCSE) represents a paradigm shift in thinking rather than merely a new technical architecture. It is based on the idea that the most valuable and enduring asset in engineering is not a specific tool, notation or model, but meaningful data that captures the intent of the system, the design decisions made, how it behaves and the evidence available. DCSE therefore treats Systems Engineering as the disciplined management of a shared semantic data space. Rather than forming isolated focal points, engineering models and documents become views, and tools become instruments that interact with, and contribute to, this shared semantic data space.

This philosophy has several implications. Firstly, DCSE insists that engineering knowledge is expressed in a way that is independent of any single tool and robust against technological change. Data and semantics must outlive file formats, tool vendors and individual projects to enable continuity across product generations and organizational transformations. Secondly, the DCSE explicitly embraces heterogeneity as the norm, expecting different disciplines, partners and lifecycle phases to use different methods and tools. The role of Systems Engineering is then to provide a coherent semantic fabric across these. Thirdly, DCSE is inherently decision-oriented; the purpose of the semantic data space is not archival completeness, but rather to enable engineers and organizations to pose and answer complex, cross-domain questions reliably. In that sense, the essence of DCSE can be summarized as follows: “Engineering as continuous, semantically grounded decision-making over an evolving, shared body of semantic data.”

4. Summary and Outlook

This paper has introduced Data-Centric Systems Engineering (DCSE) as an evolution of traditional Model-Based Systems Engineering (MBSE), motivated by the limitations observed in current industrial practice and by INCOSE’s vision for Systems Engineering in 2035. While MBSE has significantly improved traceability and rigor compared to document-centric approaches, it often remains constrained by centralized tools, limited interoperability, and a narrow focus on early lifecycle phases. In response, DCSE places semantic data at the center of Systems Engineering, treating models, documents, and tools as transient, tool-specific projections of a persistent semantic core.

The Semantic Digital Thread (SeDT) was presented as the operational backbone of DCSE. It is built on a layered Semantic Data Model (SeDM) that spans from enterprise upper ontologies down to persistence models. This architecture enables semantic integration of heterogeneous data sources, including structured enterprise systems, unstructured textual artefacts, and parameters hidden in engineering models such as 3D-CAD or Modelica. The

SeDT ensures that entities and relationships across disciplines and lifecycle stages are represented in a coherent, machine-interpretable graph. This graph is governed by explicit constraints, provenance information, and quality mechanisms. In this way, engineering data becomes a context-aware fabric that can be queried, validated, and reused across tool boundaries and organizational silos.

On this semantic foundation, the paper illustrated how DCSE addresses high-value, cross-domain use cases. The change of brake caliper material served as a representative example to demonstrate change impact analysis and cost estimation. In conventional siloed environments, such analyses rely heavily on tacit knowledge, manual searches, and local spreadsheets, making them error-prone and difficult to audit. However, within DCSE, a deterministic impact subgraph will be constructed that covers affected entities such as components, requirements, test cases, suppliers, processes, and cost elements. This graph will be then enriched with information retrieved from relevant unstructured documents. The enriched impact graph will be synthesized using Large Language Models under strict guardrails to find secondary effects, risks, and recommendations. The result is a standalone traceable decision artefact that explains technical and financial impacts, supports informed trade-offs, and captures knowledge for future reuse.

Human-in-the-loop curation ensures that newly suggested entities and relations are validated before becoming part of the impact graph and the SeDT. This pattern exemplifies how DCSE can leverage AI capabilities without compromising on governance, explainability, and engineering rigor.

Elements of this approach have already been realized in industrial settings, where schema-aware extraction from structured sources and NLP-based extraction from unstructured documents feed a governed knowledge graph that supports deterministic user queries and graph-based analyses. At the same time, the automated data extraction from engineering models and the full implementation of the guarded LLM-based synthesis are both under active development. The work presented here therefore reflects insights gained from practical implementations, as well as open research directions.

By systematically elevating semantic data to the primary asset of Systems Engineering, DCSE provides a concrete pathway toward the fully connected, intelligent, and collaborative digital engineering environment envisioned by INCOSE. It enables organizations not only to integrate their existing tools and models, but also to make their engineering decisions transparent, explainable, and repeatable across products, programs, and generations.

A key direction for future work is the development of industry standards and best practices for the upper layers of the SeDM, the Enterprise Upper Ontology, Integration Model, and

Terminology Model. While numerous standards already exist, they typically address only parts of the required semantic landscape and do not fully match the needs of a data-centric, cross-lifecycle engineering environment.

DCSE and SeDT highlight the need for shared, machine-interpretable semantics that span systems, organizations, and lifecycle stages. Establishing interoperable upper-layer models would enable data and engineering knowledge to be exchanged between companies without prescribing specific tools or implementations. Future work will therefore focus on (i) identifying gaps between existing standards and DCSE requirements, (ii) defining minimal, extensible upper-level vocabularies and relation patterns that can serve as a basis for cross-company exchange, and (iii) validating these proposals in collaborative, multi-partner engineering projects. Such standardization efforts are a prerequisite for realizing the INCOSE Vision 2035 at an ecosystem level, where systems engineering practices extend beyond single enterprises and toolchains to collaborative value networks.

In parallel, we plan to extend the evaluation of DCSE and SeDT for arising real world cases along three dimensions:

Decision quality: accuracy and completeness of generated results compared to baseline expert assessments,

Efficiency: reduction in time and manual effort that is required to perform cross-domain analyses,

Traceability and reuse: coverage of requirement-to-test and change-to-cost links, and the degree to which captured decision artefacts are reused in subsequent steps.

References

- [1] INCOSE SEH (2023). Systems Engineering Handbook: A Guide for System Life Cycle Process and Activities (5th ed.). D. D. Walden, T. M. Shortell, G. J. Roedler, B. A. Delicado, O. Mornas, Yip Y. S., and D. Endler (Eds.). San Diego, CA: International Council on Systems Engineering. Published by John Wiley & Sons, Inc.
- [2] Hooshmand, Y.; Adamenko, D.; Kunnen, S.; Köhler, P.: An approach for holistic model-based engineering of industrial plants. In: Proceedings of the 21st International Conference on Engineering Design (ICED17), Vol. 3: Product, Services and Systems Design, Vancouver, Canada, 21.-25.08.2017.
- [3] Systems Engineering Vision 2020, version 2.03, INCOSE 2007. Available at: http://oldsite.incose.org/ProductsPubs/pdf/SEVision2020_20071003_v2_03.pdf

- [4] Systems Engineering Vision 2035 - Engineering solutions for a better world, INCOSE 2021. Available at: <https://www.incose.org/docs/default-source/se-vision/incose-se-vision-2035.pdf>
- [5] McComb, D. (2018). *Software Wasteland: How the Application-Centric Mindset is Hobbling our Enterprises*. Technics Publications.
- [6] McComb, D. (2019). *The Data-Centric Revolution: Restoring Sanity to Enterprise Information Systems*. Technics Publications.
- [7] Pfenning, M. (2017), *Durchgängiges Engineering durch die Integration von PLM und MBSE*, PhD thesis, Technische Universität Kaiserslautern.
- [8] Hooshmand, Y.; Adamenko, D.; Kunnen, S.; Köhler, P. (2018), *Semi-Automatic Creation of System Models based on SysML Model Libraries*, INCOSE EMEA Sector Systems Engineering Conference, Berlin.
- [9] Lu, G. Tsinarakis, N. Sarantinoudis, G. Arampatzis, X. Zheng, and Kiritsis, D. (2024). "A semantic model-based systems engineering approach for assessing the operational performance of metal forming process," *Comput. Ind. Eng.*, vol. 190, p. 110042, 2024.
- [10] Chen, J.; Chen, Y.; Hu, Z.; Lu, J.; Zheng, X.; Zhang, H.; Kiritsis, D. (2022). A semantic ontology-based approach to support model-based systems engineering design for an aircraft prognostic health management system. *Front. Manuf. Technol.* 2022, 2, 886518.
- [11] Köhler, P., Nagarajah, A., Kunnen, S., Adamenko, D., Hooshmand, Y., and Heimsoth, A. (2019). *Unterstützung von multidisziplinären Engineering-Prozessen im Kraftwerksbau (UMEK)*. DOI: 10.2314/KXP:166775940X
- [12] Evans, E. (2003), *Domain-Driven Design: Tackling Complexity in the Heart of Software*, Boston, Addison-Wesley Professional.
- [13] Hooshmand, Y., Resch, J., Wischnewski, P., and Patil, P. (2022). From a Monolithic PLM Landscape to a Federated Domain and Data Mesh. *Proceedings of the Design Society*, 2, 713-722. DOI:10.1017/pds.2022.73



About the Author

Dr. Yousef Hooshmand specializes in Product Lifecycle Management (PLM) and Systems Engineering, with experience in automotive and energy domains. His research interests include development of federated PLM landscapes, enabling Data-Centric Systems Engineering, and semantic integration across disciplines, tools, and lifecycle stages.

LinkedIn: <https://www.linkedin.com/in/yousef-hooshmand/>